

CWE principali dello studio che hanno senso concettuale in OCaml (anche se nel tuo dataset possono comparire zero volte).

CWE ID	Nome sintetico	Applicabile a OCaml?	Note per il confronto con Python/JS
CWE-330	Random non sufficientemente sicuro	Sì	OCaml ha Random non crittografico; buona per mostrare stesso rischio ma paradigma neutro. ppl-ai-file-upload.s3.amazonaws
CWE-94	Code generation / code injection	Parzialmente	Niente eval dinamico di default; se usi DSL o interpreter custom puoi modellarla, altrimenti la usi come “quasi impossibile” in OCaml vs Python. ppl-ai-file-upload.s3.amazonaws+1
CWE-78	OS command injection	Sì	Possibile via Unix.system / Sys.command con stringhe concatenate da input; utile per mostrare come FP non basta se si chiama shell. ppl-ai-file-upload.s3.amazonaws
CWE-79	XSS	Sì ma rara	Serve contesto web + HTML generato; in OCaml web (Dream, Ocsigen) si può modellare, ma ti aspetti pochissime occorrenze, differenza forte da JS. ppl-ai-file-upload.s3.amazonaws
CWE-89	SQL injection	Sì	Richiede solo binding DB; in FP tipicamente mitigata da API tipizzate/parametrizzate, quindi probabile 0 casi nel tuo dataset. ppl-ai-file-upload.s3.amazonaws
CWE-20	Input validation impropria	Sì	Facilissima da modellare: uso diretto di int_of_string / float_of_string / input_line senza controlli. Ottima per confronto con Python. ppl-ai-file-upload.s3.amazonaws
CWE-116	Output encoding improprio	Sì	Rilevante in web/CLI; spesso non appare in piccoli snippet OCaml, quindi un buon “zero”. ppl-ai-file-upload.s3.amazonaws
CWE-134	Format / string injection	Sì	OCaml ha Printf e formati; puoi cercare pattern di concatenazione di input in formati. ppl-ai-file-upload.s3.amazonaws
CWE-772	Risorsa non rilasciata	Sì	File handler, socket, canali; FP aiuta ma non garantisce, quindi confronto interessante. ppl-ai-file-upload.s3.amazonaws
CWE-770	Risorse senza limiti / throttling	Sì	Ricorsioni o cicli che allocano liste/array senza bound da input (come nel tuo log). ppl-ai-file-upload.s3.amazonaws+1

CWE ID	Nome sintetico	Applicabile a OCaml?	Note per il confronto con Python/JS
CWE-400	Resource exhaustion / DoS	Sì	Implementabile come pattern combinato di loop/ricorsione + input. ppl-ai-file-upload.s3.amazonaws
CWE-682	Calcoli/indice errato	Sì	<code>List.nth</code> , <code>Array.get</code> , <code>String.get</code> senza check; l'hai già visto nei tuoi snippet. ppl-ai-file-upload.s3.amazonaws+1
CWE-327	Crypto debole	Sì	Se compaiono librerie crypto; in OCaml meno comuni ma concettualmente valide. ppl-ai-file-upload.s3.amazonaws
CWE-295	Certificati non validati	Sì	Solo se usi librerie TLS/HTTP; spesso 0 casi nel dataset, buon “zero strutturale”. ppl-ai-file-upload.s3.amazonaws
CWE-502	Deserializzazione non sicura	Sì	Con Marshal o librerie JSON/YAML non sicure; in OCaml meno tipico che in OOP/Java. ppl-ai-file-upload.s3.amazonaws+1
CWE-259 /798	Credenziali hard-coded	Sì	Lingua-agnostica; se appaiono, puoi dire che paradigma non protegge da “secret management”. ppl-ai-file-upload.s3.amazonaws
CWE-312 /215	Exposure informazioni sensibili	Sì	Loggare cose sensibili, messaggi d'errore verbosi: linguaggio-agnostico. ppl-ai-file-upload.s3.amazonaws
CWE-396 /617	Gestione errori debole	Sì	Eccezioni non gestite, <code>failwith</code> in punti critici, ecc. ppl-ai-file-upload.s3.amazonaws
CWE-563 /561	Variabili inutilizzate / dead code	Sì	Più “code smell” che security, ma il paper le include; puoi citarle come differenza di stile FP. ppl-ai-file-upload.s3.amazonaws

CWE non compatibili

CWE (famiglia)	Compatibilità OCaml	Motivo strutturale dell'esclusione
Varianti di eval / dynamic code execution (es. sotto-casi di CWE-94, CWE-95 nel paper)	No (in OCaml puro)	Richiedono esecuzione di stringhe come codice (eval, exec, Function(. . .) in JS, eval/exec in Python). OCaml non espone un eval del sorgente a runtime nel linguaggio standard; per avere qualcosa di simile servono interpreti/DSL custom, non presenti "di default" negli snippet. ppl-ai-file-upload.s3.amazonaws+1
CWE basate su reflection OO avanzata (metodi invocati per nome, dynamic dispatch riflessivo)	No nel modello che usi	Il paper lavora su Python/JS con oggetti dinamici e reflection forte; molte debolezze nascono da invocazioni di metodi per nome, attribute access dinamico, monkey-patching. OCaml ha OO ma senza reflection simile, e nel tuo dataset lavori su moduli/funzioni, non su grandi gerarchie OOP dinamiche. ppl-ai-file-upload.s3.amazonaws
CWE tipiche di pointer / buffer C (es. buffer overflow puro, pointer arithmetic)	No in OCaml puro (sì solo via FFI)	Queste debolezze presuppongono gestione manuale di memoria e puntatori (C/C++). OCaml gestisce memoria tramite GC e non espone pointer arithmetic nel codice normale; consideri gli snippet solo OCaml, quindi escludi CWE che richiedono accesso diretto a buffer C. www-verimag.imag+1
CWE di memory management manuale (use-after-free, double free)	No in OCaml puro	Richiedono API esplicite di alloc/free. In OCaml non liberi manualmente, quindi il pattern non è esprimibile senza C stubs. ocaml
CWE che dipendono da riflessione su stack / frame locali	No	Alcune CWE in ambienti dinamici sfruttano introspezione sullo stack o sui frame (ispezione/modifica di variabili locali via reflection). OCaml non offre questo tipo di metaprogrammazione a runtime nel linguaggio di base. ocaml

Assenza di eval dinamico “alla Python/JS”

- Molte vulnerabilità del paper ruotano attorno a `eval/exec` o costruzione dinamica di codice (CWE-94, CWE-95, code injection).
-
- OCaml standard non prevede l'esecuzione diretta di stringhe come codice; se vuoi qualcosa di simile devi costruire un interprete o una DSL su misura.
-
- Quindi: *l'intera superficie di attacco “inietto stringa → viene eseguita come codice” è assente nel modello base di OCaml, mentre è naturale in Python/JS.*
- **Niente pointer arithmetic e free manuale**
- Diverse CWE classiche legate a buffer e puntatori (overflow, use-after-free, double-free) presuppongono gestione esplicita della memoria.
-
- In OCaml la memoria è gestita dal garbage collector e non hai pointer arithmetic nel codice normale: questi pattern non si possono scrivere senza uscire verso C.
-
- Quindi: *una classe intera di CWE “a basso livello” è strutturalmente esclusa dal sotto-insieme di codice che stai studiando.*
- **Reflection/OOP dinamico molto ridotti**
- Molte debolezze sugli oggetti nei linguaggi del paper (Python/JS) dipendono da oggetti e metodi risolti per nome a runtime, monkey-patching, introspezione aggressiva.
-
- OCaml ha OO, ma l'uso tipico è molto più statico e il tuo dataset è principalmente funzionale: niente dinamismo stile `getattr(obj, name)` o `obj[name]()` da stringa.
-
- Quindi: *alcune CWE legate a reflection dinamica semplicemente non hanno un pattern naturale in OCaml.*
- **Che cosa puoi dire in tesi (frase “forte” ma onesta)**
Una formulazione che puoi usare (parafrasi accademica) è:

“Rispetto a Python e JavaScript, OCaml offre un modello di linguaggio che elimina intere superfici di attacco considerate nello studio originale, in particolare le debolezze basate su esecuzione dinamica di codice (`eval`), reflection OO intensiva e gestione manuale della memoria. In questo lavoro, tali CWE sono classificate come strutturalmente non applicabili al sotto-insieme di codice OCaml analizzato. L'assenza di queste categorie non è un artefatto del dataset, ma una conseguenza diretta delle scelte di paradigma e di design del linguaggio.”